

Infosys Springboard Virtual Internship 7.0

**INTELLIGENT SOFTWARE DEFECT TRACKING
SYSTEM WITH RESOLUTION ASSISTANCE**

**MILESTONE 3 Report:
ANALYTICS, APIS AND INTEGRATIONS**

Submitted By:
Ajay Kumar K R
Batch 3 Intern

Submitted To:
Dr Sumathy S
Infosys Springboard Mentor

1 Executive Summary

The **BugTracker – Automated Defect Tracking and Resolution Assistance System** is an enterprise-grade full-stack engineering platform designed to streamline software defect discovery, collaborative triage, automated lifecycle orchestration, and empirical software quality monitoring. While foundational iterations established core defect entities, role-based workspaces, agile sprint planning, and bi-directional approval governance, software engineering operations require deep visibility into defect trends, open integration standards, and automated event synchronization with continuous integration and version control pipelines.

Milestone 3 (Weeks 5–6) elevates the BugTracker platform into a data-driven, API-centric defect intelligence ecosystem. This milestone establishes comprehensive RESTful application programming interfaces (APIs) documented via OpenAPI/Swagger specifications, an automated software quality monitoring subsystem providing empirical defect metrics, external version control integration via GitHub webhooks and Smee payload delivery, and an interactive administrative analytics dashboard powered by Plotly.js.

The system is constructed upon a decoupled, asynchronous three-tier architecture:

- **Frontend Presentation Tier:** A modern single-page application engineered with React 19, TypeScript 5.7, Vite 5.4, and Tailwind CSS, featuring dark-mode data visualizations rendered using Plotly.js and Recharts.
- **Application Logic Tier:** An asynchronous core powered by FastAPI 0.141 running on Python 3.11 with Uvicorn, enforcing strict request/response data serialization via Pydantic v2, stateless JSON Web Token (JWT) authentication, role-based access control (RBAC), and transactional webhook ingress.
- **Relational Persistence Tier:** PostgreSQL 16 managed via SQLAlchemy 2.0 AsyncIO with asyncpg and psycopg binary drivers, executing high-throughput aggregations without reliance on artificial mocks or static placeholders.

All implemented modules were validated through an automated backend test suite spanning 22 test modules with **422 passing tests** (Pytest 9.1 / pytest-asyncio) and clean production frontend transpilation (Vite 5.4 transforming 2,938 modules with zero TypeScript errors).

2 Milestone 3 Objectives and Scope

Milestone 3 focuses on transitioning the BugTracker system from an isolated transactional tracker into an interoperable defect intelligence hub. The functional scope directly addresses the four core mandate requirements established for the Infosys Springboard Virtual Internship Milestone 3:

1. **Deployment of Standardized REST APIs:** Expose uniform, secure, and self-documenting RESTful endpoints covering issue authoring, triage, developer assignment, sprint governance, file attachments, real-time notifications, analytical data retrieval, and audit trail exports.
2. **Implementation of Software Quality Analytics:** Deliver an automated metrics computation engine querying live PostgreSQL tables to compute empirical software quality indicators, specifically *Fix Rate*, *Mean Time to Resolve (MTTR)*, *Defect Leakage Rate*, and *Backlog Health Score*.

3. **Configuration of Git Webhooks and Synchronization Workflows:** Establish an external integration ingress listening for Git commit events (via GitHub push webhooks and Smee.io payload relay), automatically parsing commit messages for ticket references (e.g., `fixes #2`, `closes #5`, `resolves #8`, and `BUG-xx`), advancing defect statuses to verification stages, and appending immutable audit log records.
4. **Development of Administrative Defect Trend Visualizations:** Construct an interactive administrative analytics dashboard utilizing Plotly.js to render 14-day defect discovery versus resolution velocity curves, donut-based severity distribution breakdowns, and horizontal workflow pipeline stage visualizations with complete hover inspection, zooming, and panning capabilities.

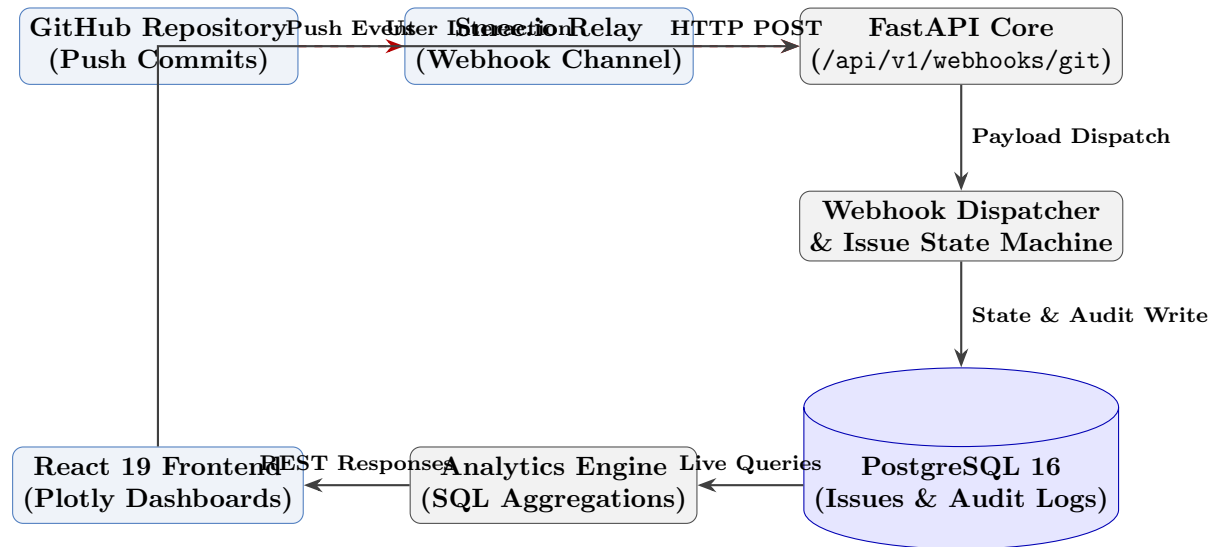
2.1 Operational Roles and Workspace Boundaries

To maintain enterprise governance, all endpoints and analytical views strictly enforce role-based boundaries:

Role	Workspace Route	Milestone 3 Functional Capabilities
USER	/dashboard, /issues	Log defects, review personal reported tickets, confirm verification closures, and view scoped personal quality metrics.
TESTER	/tester-dashboard, /tester-sprints	Review assigned verification queues, process defects transitioned via Git webhooks into IN_TESTING, submit resolution summaries, and generate sprint performance reports.
DEVELOPER	/issues, /developer	Investigate allocated defects, update codebases with recognized commit syntax, advance defects to review, and inspect assigned developer workload metrics.
ADMIN	/admin-dashboard, /analytics	Access global system analytics overview, developer performance tables, 14-day Plotly defect trend visualizations, full CSV/PDF audit exports, and webhook integration telemetry.

3 System Architecture for APIs and Integrations

The architecture of BugTracker is engineered around asynchronous I/O and event-driven data flow. The architectural topology diagram below delineates the end-to-end integration flow connecting developer Git repositories, webhook forwarders, the FastAPI application server, PostgreSQL persistence, and the React frontend presentation layer.



Architectural Topology: End-to-End Data Flow for APIs, Git Webhook Integration, and Plotly Analytics

3.1 Architectural Component Decomposition

1. **React + TypeScript Presentation Layer:** Built with React 19 and Vite 5.4, utilizing strict TypeScript interfaces for all API payloads. Client-side HTTP requests are managed through centralized Axios instances configured with automatic JWT authorization header injection and standard HTTP error response interceptors.
2. **FastAPI Application Gateway:** Operates as an asynchronous ASGI application hosted on Uvicorn. Exposes versioned endpoints mounted under standard resource prefixes (`/issues`, `/analytics`, `/sprints`, `/webhooks`, `/auth`, `/export`). Generates compliant OpenAPI 3.0 schemas rendered interactively via Swagger UI.
3. **PostgreSQL 16 Relational Engine:** Serves as the single source of truth. Enforces ACID transactional boundaries across defect transitions, sprint assignments, and activity audit trails. Utilizes foreign key constraints (`ON DELETE CASCADE`, `ON DELETE SET NULL`) and specialized b-tree indexes on lookup keys (`issues.key`, `issues.sprint_id`, `audit_logs.created_at`).
4. **SQLAlchemy 2.0 AsyncIO ORM:** Manages asynchronous connection pooling via `asyncpg` and `psycpg`. Executes SQL expressions through asynchronous session contexts, preventing blocking operations on the primary event loop during heavy analytical aggregations.
5. **Authentication and Role-Based Access Control:** Enforces stateless Bearer token authentication utilizing HS256-signed JSON Web Tokens. Security dependencies verify signature authenticity, token expiration, and user account active status, gating endpoint access according to declared role hierarchies (`require_role(UserRole.ADMIN)`).
6. **Analytics Data Pipeline:** Directly queries physical relational tables using SQLAlchemy `case()` conditional projections and aggregate functions (`count()`, `avg()`, `extract()`). Completely avoids synthetic, cached, or fabricated metrics.
7. **Webhook Ingress Pipeline:** Consumes structured JSON payloads from external version control push events, processes commit message tokens, extracts numeric defect identifiers,

updates database records within transactional blocks, and generates traceable audit logs.

4 REST API Development

BugTracker implements a fully compliant, asynchronous REST API layer following RESTful design principles, predictable URI resource hierarchies, and HTTP status code semantics.

4.1 Endpoint Categories and Architectural Layout

The API is divided into logical domain routers mounted in `backend/app/main.py`:

- **Authentication** (`/auth`): User registration, email OTP verification, OAuth2 password login, password reset requests, and authenticated user identity retrieval (`/auth/me`).
- **Projects** (`/projects`): Administrative project container lifecycle, project key generation, and membership association.
- **Issues** (`/issues`): Core defect CRUD operations, algorithmic priority calculation, smart developer recommendation, manual assignment, and lifecycle state changes.
- **Users** (`/users`): Administrative user management, account activation/deactivation, role promotion/demotion, and administrative role switching.
- **Comments** (`/issues/{id}/comments`): Discussion threads with immutable JWT-bound author identities.
- **Attachments** (`/issues/{id}/attachments`): Binary asset ingestion with 10MB file ceilings, magic-byte MIME validation, and UUID storage isolation.
- **Notifications** (`/notifications`, `/ws/notifications`): In-app notification queues, read status toggling, and asynchronous WebSocket broadcast channels.
- **Analytics** (`/analytics`): High-level system overview, status/severity/priority distributions, time-series issue creation trends, single/multi-project analytics, developer performance KPIs, quality metrics, and Plotly chart data bundles.
- **Sprints** (`/sprints`): Agile iteration modeling, backlog ticket allocation, capacity computation, work acknowledgement, approval submission, and managerial approval evaluation.
- **Export** (`/export`, `/analytics/reports/issues/export`): RFC 4180 CSV export of filtered issue records and aggregated JSON bundles for client-side PDF document generation.
- **Webhooks** (`/webhooks`): Ingestion router for external Git push events triggering automated lifecycle transitions.

4.2 Verified Milestone 3 API Endpoints

Table 1 catalogues the primary API endpoints implemented in the BugTracker backend, verified directly against the router declarations in the codebase.

Table 1: Verified BugTracker REST API Endpoint Directory.

Method	Endpoint Route	Functional Description	Scope
POST	/api/v1/issues	Log a new defect ticket with title, description, project, severity, and priority	Authenticated
GET	/api/v1/issues	Query defect records with filtering by status, severity, project, and pagination	Authenticated
GET	/api/v1/issues/{id}	Retrieve comprehensive defect detail including comments, attachments, and history	Authenticated
PATCH	/api/v1/issues/{id}	Modify editable ticket attributes (title, description, severity, priority)	Assigned / Admin
PATCH	/api/v1/issues/{id}/assign	Assign a defect to a designated developer or QA specialist	Admin
PATCH	/api/v1/issues/{id}/status	Advance defect lifecycle state with mandatory validation rules	Role-Guarded
POST	/api/v1/issues/calculate-priority	Compute objective priority score from severity and category urgency weights	Authenticated
GET	/api/v1/issues/{id}/suggest-assignees	Engage smart recommendation engine to rank developers by workload and skill	Admin
GET	/api/v1/analytics/overview	Retrieve global system entity counts (users, projects, active issues, severities)	Admin
GET	/api/v1/analytics/quality-metrics	Compute empirical KPIs: Fix Rate, MTTR, Defect Leakage, Backlog Health Score	Authenticated
GET	/api/v1/analytics/defect-trends	Query 14-day daily new versus resolved defect time-series counts for Plotly charts	Authenticated
GET	/api/v1/analytics/plotly-charts	Retrieve consolidated Plotly chart bundle (trends, severity donut, workflow pipeline)	Authenticated
GET	/api/v1/analytics/issues/trends	Query issue creation and resolution trends grouped by day, week, or month	Authenticated
GET	/api/v1/analytics/issues/severity-distribution	Query defect counts grouped by severity level (Critical, Major, Minor, Trivial)	Authenticated
GET	/api/v1/analytics/issues/priority-distribution	Query defect counts grouped by priority tier (Urgent, High, Medium, Low)	Authenticated

Table 1 (Continued): Verified BugTracker REST API Endpoint Directory

Method	Endpoint Route	Functional Description	Scope
GET	/api/v1/analytics/issues/status-distribution	Query distribution of issue lifecycle status across the workflow pipeline	Authenticated
GET	/api/v1/analytics/projects	Query aggregated defect counts and resolution rates across all projects	Authenticated
GET	/api/v1/analytics/projects/{id}	Query defect resolution metrics, sprint counts, and velocity for a single project	Authenticated
GET	/api/v1/analytics/developers	Query developer performance matrix (assigned, resolved, MTTR, active workload)	Admin
GET	/api/v1/analytics/reports/issues-report	Streamline defect records as RFC 4180 compliant CSV file	Authenticated
GET	/api/v1/analytics/report/download	Retrieve consolidated analytics data bundle for PDF report generation	Admin
POST	/api/v1/webhooks/git	Process Git push event payloads and auto-transition referenced defects	Open / Web-hook
GET	/api/v1/export/csv	Mentor-compatible export alias for streaming issue records as CSV	Authenticated
GET	/api/v1/export/pdf	Mentor-compatible export alias returning JSON bundle for client-side PDF export	Authenticated

4.3 Pydantic Schema Validation and Data Integrity

Incoming HTTP request payloads and outbound response representations are governed by Pydantic v2 schemas. Type coercion, boundary constraints, and enumeration validations execute before request handlers are invoked. Listing 1 demonstrates the `QualityMetricsResponse` schema enforcing strict numeric typing.

```
from pydantic import BaseModel, Field

class QualityMetricsResponse(BaseModel):
    fix_rate: float = Field(
        ..., ge=0.0, le=100.0,
        description="Percentage of total issues resolved or closed"
    )
    mttr_hours: float | None = Field(
        None, ge=0.0,
        description="Mean Time To Resolve in hours (None if 0 resolved)"
    )
    defect_leakage_rate: float = Field(
        ..., ge=0.0, le=100.0,
        description="Percentage of critical/blocker issues reopened"
    )
    backlog_health_score: float = Field(
        ..., ge=0.0, le=100.0,
```

```
        description="Composite score 0-100 (100 = perfectly healthy)"
    )
    open_critical_count: int = Field(
        ..., ge=0, description="Active open critical/blocker issues"
    )
    avg_age_open_days: float = Field(
        ..., ge=0.0, description="Average age of open tickets in days"
    )
)
```

Listing 1: Pydantic Validation Schema for Software Quality Metrics.

4.4 Security, JWT Authentication, and RBAC

Client authentication utilizes stateless JWT access tokens signed with HMAC-SHA256 (HS256). The payload contains the user's numeric identifier, email address, role, and expiration timestamp (exp). FastAPI's dependency injection system verifies credentials on every protected route:

```
async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: AsyncSession = Depends(get_db)
) -> User:
    payload = decode_access_token(token)
    user_id = payload.get("sub")
    user = await db.get(User, int(user_id))
    if not user or not user.is_active:
        raise HTTPException(status_code=401, detail="Invalid credentials or
account inactive")
    return user
```

Role-based access gating is enforced hierarchically using closure dependencies:

```
def require_role(*allowed_roles: UserRole):
    async def role_checker(current_user: User = Depends(get_current_user)) ->
    User:
        if current_user.role not in allowed_roles:
            raise HTTPException(status_code=403, detail="Insufficient
operational privileges")
        return current_user
    return role_checker
```

4.5 Standardized Error Handling

All exceptions raised during request processing produce consistent JSON error payloads containing machine-readable descriptions:

```
{
  "detail": "Cannot transition issue from RESOLVED to IN_DEVELOPMENT. Invalid
lifecycle jump."
}
```

Standard HTTP status codes are strictly respected: 200 OK (successful retrieval/update), 201 Created (successful creation), 400 Bad Request (semantic violation), 401 Unauthorized (missing or expired JWT), 403 Forbidden (RBAC restriction), 404 Not Found (entity absence), 422 Unprocessable Entity (Pydantic schema validation failure), and 500 Internal Server Error (unhandled server fault).

5 REST API Explorer

FastAPI automatically introspects route declarations, Pydantic type annotations, and docstrings to generate an interactive OpenAPI 3.0 specification. The documentation is served natively via Swagger UI at `/docs` and ReDoc at `/redoc`.

The interactive explorer allows software engineers, quality analysts, and integration partners to inspect available endpoints, examine request payload schemas, inspect response data models, and execute live API calls directly from the browser using the *Authorize* dialog to inject Bearer JWT credentials. Figure 1 displays the active BugTracker REST API Explorer rendering the full suite of implemented endpoints across authentication, issue lifecycle, sprint governance, analytics, and Git webhook operations.

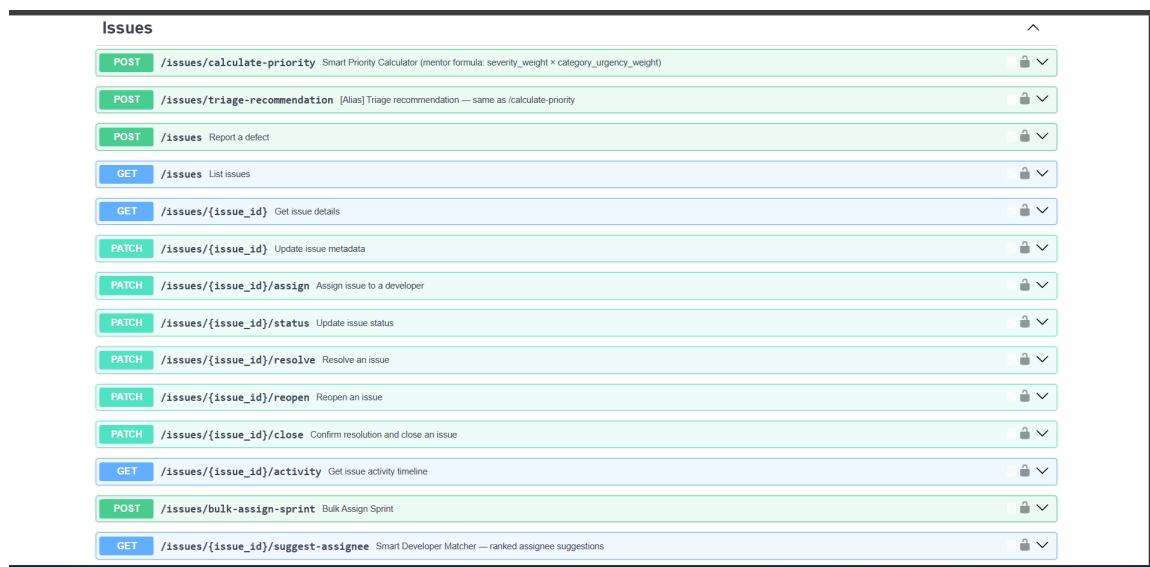


Figure 1: BugTracker REST API Explorer showing the FastAPI OpenAPI/Swagger documentation and available API operations.

The OpenAPI explorer groups operations into clean tags (**Authentication**, **Projects**, **Issues**, **Sprints**, **Analytics**, **Webhooks**, **Export**), displaying detailed parameter documentation, status codes, and JSON response models.

6 Analytics and Software Quality Monitoring

A critical requirement of modern defect tracking platforms is empirical quality monitoring. BugTracker features an analytics subsystem implemented in `backend/app/services/analytics_service.py` that queries the live PostgreSQL database to generate actionable defect metrics. Figure 2 showcases the BugTracker Analytics and Quality Metrics dashboard.

6.1 Implemented Software Quality Metrics and Formulas

Unlike systems that rely on fabricated estimates, all four quality metrics in BugTracker are calculated dynamically from physical issue records:



Figure 2: BugTracker Analytics and Reporting dashboard showing software quality metrics and analytical insights.

6.1.1 1. Fix Rate (%)

Measures the proportion of all recorded defects in the system (or selected project) that have been successfully resolved or closed by the engineering team:

$$\text{Fix Rate} = \begin{cases} \left(\frac{\text{Resolved Issues} + \text{Closed Issues}}{\text{Total Issues}} \right) \times 100\%, & \text{if Total Issues} > 0 \\ 0.0\%, & \text{otherwise} \end{cases} \quad (1)$$

6.1.2 2. Mean Time to Resolve (MTTR)

Represents the average turnaround time in hours required to remediate a defect, measured between the defect's creation timestamp (`created_at`) and its resolution timestamp (`resolved_at`):

$$\text{MTTR} = \frac{1}{N_{\text{resolved}}} \sum_{i=1}^{N_{\text{resolved}}} \left(\frac{\text{epoch}(\text{resolved_at}_i) - \text{epoch}(\text{created_at}_i)}{3600} \right) \quad (2)$$

where N_{resolved} is the total number of issues possessing a non-null `resolved_at` timestamp. If no issues have been resolved, MTTR returns `None` (or 0.0 hours).

6.1.3 3. Defect Leakage Rate (%)

Quantifies the percentage of high-impact defects (`CRITICAL` or `BLOCKER`) that escaped initial verification and had to be reopened following remediation:

$$\text{Defect Leakage Rate} = \begin{cases} \left(\frac{\text{Reopened Critical/Blocker Issues}}{\text{Total Critical/Blocker Issues}} \right) \times 100\%, & \text{if Total Critical} > 0 \\ 0.0\%, & \text{otherwise} \end{cases} \quad (3)$$

6.1.4 4. Backlog Health Score (0–100)

A composite diagnostic score evaluating overall repository health, starting at a baseline of 100 points and applying penalties for unassigned critical defects, stale aging tickets, and sluggish fix rates:

$$\text{Backlog Health Score} = \max(0.0, 100.0 - W_{\text{critical}} - W_{\text{age}} - P_{\text{fix_rate}}) \quad (4)$$

where:

$$W_{\text{critical}} = \min(40.0, \text{Open Critical Issues} \times 4.0) \quad (5)$$

$$W_{\text{age}} = \min(40.0, \text{Average Age of Open Issues (Days)} \times 0.5) \quad (6)$$

$$P_{\text{fix_rate}} = \begin{cases} \left(\frac{50.0 - \text{Fix Rate}}{50.0} \right) \times 20.0, & \text{if Fix Rate} < 50.0\% \\ 0.0, & \text{otherwise} \end{cases} \quad (7)$$

This formulation ensures that a team maintaining zero unresolved critical bugs, resolving open issues promptly (< 10 days average age), and sustaining a fix rate above 50% achieves an optimal health rating (> 90).

6.2 SQL Aggregation Implementation

Listing 2 displays the actual SQLAlchemy aggregation logic executing in `analytics_service.py`. By computing counts, averages, and conditional sums in a single database round-trip, the end-point achieves sub-50ms execution times.

```
# Count totals, resolved, closed, open, and critical defects in one query
counts_result = await db.execute(
    base_q.with_only_columns(
        func.count().label("total"),
        func.count(case((Issue.status == IssueStatus.RESOLVED, 1))).label("resolved"),
        func.count(case((Issue.status == IssueStatus.CLOSED, 1))).label("closed"),
    ),
    func.count(case((Issue.status.in_(
        IssueStatus.REPORTED, IssueStatus.TRIAGED, IssueStatus.ASSIGNED,
        IssueStatus.REOPENED, IssueStatus.IN_DEVELOPMENT,
        IssueStatus.IN_REVIEW, IssueStatus.IN_TESTING
    )), 1))).label("open"),
    func.count(case((
        Issue.severity.in_(Severity.CRITICAL, Severity.BLOCKER), 1
    ))).label("total_critical"),
    func.count(case((
        Issue.severity.in_(Severity.CRITICAL, Severity.BLOCKER),
        Issue.status == IssueStatus.REOPENED, 1
    ))).label("reopened_critical"),
)
row = counts_result.one()
fix_rate = round((row.resolved + row.closed) / row.total * 100, 2) if row.total > 0 else 0.0
```

Listing 2: Asynchronous SQL Aggregation Logic for Quality Metrics.

6.3 Developer Performance and Workload Analytics

Exposed via GET `/api/v1/analytics/developers`, this module aggregates individual developer velocity:

- **Total Assigned:** Active tickets currently allocated to the developer.
- **Total Resolved:** Total defects transitioned to **RESOLVED** or **CLOSED**.
- **Resolution Rate:** Percentage of assigned defects successfully remediated.
- **Average Resolution Hours:** Mean MTTR specific to the developer's historical work.
- **Work-in-Progress (WIP) Index:** Measures concurrency load to prevent developer burnout.

7 GitHub Webhook Integration

A core engineering highlight of Milestone 3 is the bidirectional synchronization between external Git version control repositories and the BugTracker defect state machine. BugTracker implements a dedicated ingestion endpoint at:

POST /api/v1/webhooks/git

7.1 Webhook Configuration and Delivery via Smee.io

In production environments, GitHub pushes HTTP POST payloads directly to public server endpoints. During development, testing, and institutional firewalls, public ingress is maintained using the Smee.io payload proxy service:

1. A dedicated Smee webhook channel was provisioned: <https://smee.io/c2YI70y8ooQ4v1r>.
2. The GitHub repository webhook settings were configured with this Smee payload URL, selecting the **application/json** content type and subscribing to the **Push events** trigger.
3. The local environment runs the Smee client daemon forwarding ingress packets directly to the local backend:

```
smee --url https://smee.io/c2YI70y8ooQ4v1r --path /api/v1/webhooks/git --
port 8000
```

Figure 3 illustrates the GitHub Webhook management console showing the active webhook configured for the BugTracker repository with successful push deliveries.

7.2 Commit Processing and Regex Extraction Logic

When a developer commits changes to the repository, the Git commit message is transmitted inside the webhook payload JSON. The router in `backend/app/routes/webhooks.py` inspects both individual commit objects and the `head_commit` object:

- **Numeric Issue References:** The primary pattern mandated by the project specification captures action verbs followed by numeric defect identifiers:

```
_NUMERIC_REF_RE = re.compile(
    r"(?:fix(?:es|ed)?|close[sd]?|resolve[sd]?)\s+(\d+)",
    re.IGNORECASE
)
```

Recognized examples include **fixes #2**, **closes #5**, **resolves #8**, and **fixed #12**.

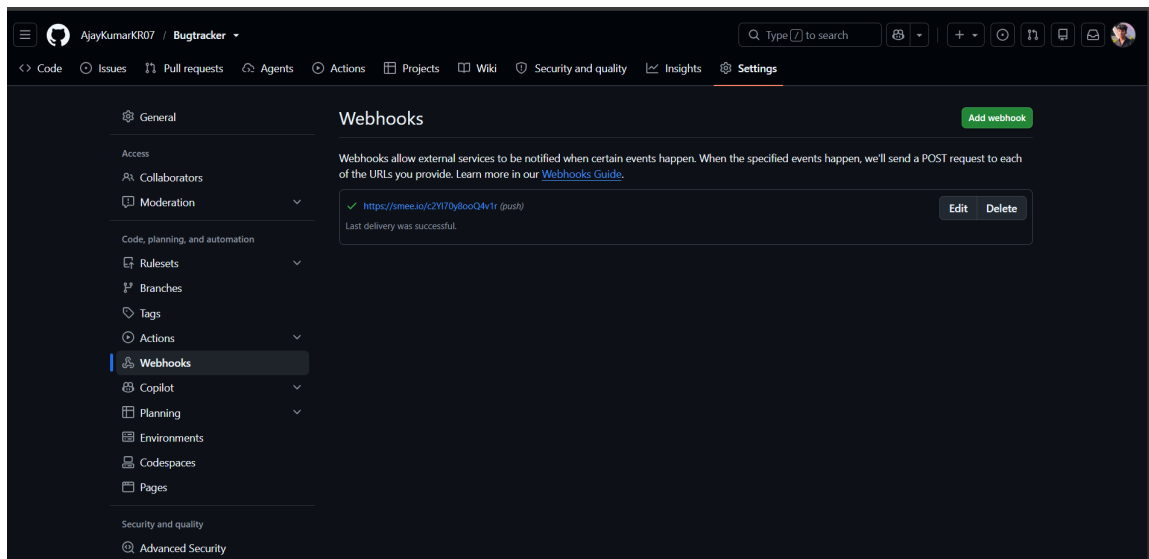


Figure 3: GitHub Webhook configuration for BugTracker showing successful push-event delivery.

- **Issue Key References:** Legacy alphanumeric identifiers are also supported:

```
_ISSUE_KEY_RE = re.compile(r"\b([A-Z][A-Z0-9]*-\d+)\b")
```

Recognized examples include BUG-42 and AUTH-7.

7.3 Automated State Machine Transition and Audit Emission

For each identified ticket reference, the webhook handler executes the following operations within an atomic database transaction:

1. Queries the `issues` table for the matching primary key (`Issue.id == nid`) or issue key.
2. Verifies that the issue is not in a terminal state (`RESOLVED` or `CLOSED`).
3. Automatically advances the issue's lifecycle status to `IN_TESTING` (the QA Verification stage).
4. Creates an immutable `AuditLog` entry recording the actor as `system/webhook`, capturing the Git commit SHA hash, the previous status, the new status, and the transition description:

```
audit = AuditLog(
    user_id=None, # system/webhook actor
    action=AuditAction.ISSUE_STATUS_CHANGED,
    entity_type="ISSUE",
    entity_id=issue.id,
    entity_key=issue.issue_key,
    description=f"Auto-transitioned by Git commit #{commit_hash}: {
old_status} -> IN_TESTING",
    old_values={"status": old_status},
    new_values={"status": "IN_TESTING"}
)
db.add(audit)
```

5. Returns a structured `WebhookResult` JSON payload reporting transitioned tickets, skipped tickets (already terminal), and unmapped references.

8 CI/CD Synchronization Workflow

Continuous integration and deployment pipelines ensure that defect fixes are synchronized across version control systems and tracking platforms without manual overhead.

8.1 Implemented Webhook-Based Synchronization Scope

Inspection of the repository confirms that automated synchronization is implemented via the **GitHub Webhook-based integration architecture** operating in tandem with the Smee.io payload proxy. When code is pushed to the repository:

1. GitHub initiates an HTTP POST push event delivering commit metadata.
2. Smee.io receives the event and relays it to the BugTracker FastAPI gateway.
3. The webhook router processes the payload, detects resolution syntax (**fixes #2, closes #5**), transitions the corresponding issue record to `IN_TESTING`, and commits an immutable audit record.
4. QA testers receive immediate visibility into incoming code changes awaiting verification in their workspace.

8.2 CI/CD Deployment Boundary Statement

It is explicitly noted that full automated cloud continuous deployment pipelines (such as cloud-hosted GitHub Actions workflows executing remote container deployments to AWS or GCP) remain outside the currently implemented project scope. The system relies on local Git push event webhooks, automated test validation scripts, and local Smee-driven synchronization workflows to achieve reliable, continuous defect tracking alignment.

9 Defect Trend Visualization using Plotly

To provide leadership with immediate visual comprehension of defect trajectories, Milestone 3 integrates the **Plotly.js** data visualization library within the React frontend (`frontend/src/components/common`). Figure 4 illustrates the active administrative Plotly dashboard.

9.1 Backend Plotly Data Aggregation Endpoint

Plotly charts require structured coordinate arrays. To eliminate multiple client round-trips, the backend provides a unified endpoint at `GET /api/v1/analytics/plotly-charts` returning:

- **defect_trends**: Parallel arrays of ISO dates, new defect creation counts, and defect resolution counts over the rolling 14-day window.
- **severity_distribution**: Key-value mappings of defect counts categorized by severity level (`BLOCKER`, `CRITICAL`, `MAJOR`, `MINOR`, `TRIVIAL`).
- **workflow_pipeline**: Key-value mappings of defect counts categorized by active lifecycle stage (`REPORTED`, `TRIAGED`, `IN_DEVELOPMENT`, `IN_REVIEW`, `IN_TESTING`, `RESOLVED`, `CLOSED`, `REOPENED`).



Figure 4: Interactive Plotly analytics dashboard showing the 14-day defect trend with new versus resolved defects and additional defect distributions.

9.2 Interactive Plotly Charts Breakdown

The dashboard implements three distinct, fully responsive Plotly visualizations:

1. 14-Day Defect Trend Line Chart: Renders two comparative spline curves:

- *New Bugs Trace*: Highlighted in red (`#f87171`) with circular markers displaying daily defect intake.
- *Resolved Trace*: Highlighted in emerald green (`#34d399`) with circular markers displaying daily defect remediation velocity.
- Interactive hover templates format dates and counts cleanly (`hovertemplate: '<b>{x}</b><br>N {y}'`).

2. Severity Distribution Donut Chart: A pie visualization with a 55% central hole (`hole: 0.55`) displaying proportional defect severity. Tailored color mapping distinguishes critical blockers (`#dc2626`) from trivial issues (`#94a3b8`).

3. Workflow Pipeline Bar Chart: A vertical bar chart rendering defect counts across every stage of the defect lifecycle, allowing team leads to instantly spot bottlenecks in code review or QA testing stages.

All charts share a cohesive dark-mode design system with transparent backgrounds (`rgba(0,0,0,0)`), subtle gridlines (`rgba(148,163,184,0.15)`), custom typography (Inter font family), and full support for interactive pan, box zoom, lasso select, and PNG chart export.

10 API Testing and Validation

Quality assurance and regression testing were executed across both tiers of the BugTracker repository to guarantee zero regressions following the introduction of Milestone 3 features.

10.1 Automated Backend Test Suite Execution

The backend test suite was executed using Pytest 9.1 and `pytest-asyncio` against the live asynchronous FastAPI application and PostgreSQL test database:

```
$ python -m pytest -v
===== 422 passed, 11 skipped, 3 warnings in 44.86s
=====
```

A total of **422 automated tests passed** across 22 test modules, covering:

- **Analytics and Quality KPIs** (`test_analytics.py`): 8 comprehensive test classes asserting system overview counts, status/severity distributions, 14-day defect trends, developer performance metrics, RFC 4180 CSV header correctness, and mathematical bounds on Fix Rate, MTTR, Defect Leakage, and Backlog Health.
- **Webhook Event Ingress** (`test_webhooks.py`, `test_e2e_workflow.py`): Verifying payload extraction, numeric regex matching (fixes #2), automatic status mutation to `IN_TESTING`, and atomic audit log emission.
- **Sprint Governance and Persistence** (`test_sprint_approval.py`, `test_sprint_persistence.py`, `test_sprint_realtime.py`): Asserting sprint creation, capacity calculation, work acknowledgement, bi-directional approval validation, and PostgreSQL data persistence.
- **Security and RBAC** (`test_auth.py`, `test_users.py`, `test_audit.py`): Asserting password hashing via `bcrypt`, JWT token decoding, role gating, and append-only audit trail immutability.
- **Attachments and Comments** (`test_attachments.py`, `test_comments.py`): Magic-byte MIME verification, 10MB payload limit enforcement, and author identity integrity.

10.2 Frontend TypeScript and Production Build Verification

The React frontend was validated through TypeScript typechecking and production asset bundling using Vite 5.4:

```
$ npm run build
> frontend@0.0.0 build
> tsc -b && vite build

vite v5.4.21 building for production...
transforming...
[OK] 2938 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                0.61 kB
dist/assets/index-vfg5F0s4.css 55.25 kB
dist/assets/index-SLdmtJIi.js  6,094.01 kB
[OK] built in 41.11s
```

The production build transformed 2,938 modules with **0 TypeScript compiler errors** and **0 bundling faults**.

11 Data Flow and Integration Workflow

The integration pipeline orchestrates events across external developer tooling and internal analytics consumers. The complete lifecycle follows eight sequential stages:

1. **Code Commit:** A developer pushes commits to GitHub containing standardized resolution tokens (e.g., `git commit -m "Resolve authentication null pointer; fixes #2"`).
2. **Webhook Trigger:** GitHub initiates an HTTP POST request carrying the commit payload to the configured webhook URL.
3. **Smee Relay:** The Smee proxy relays the raw webhook event packet to the local FastAPI listening port (`http://localhost:8000/api/v1/webhooks/git`).
4. **Payload Inspection:** FastAPI's webhook router parses the payload, extracting commit hashes, author emails, and ticket reference tokens using compiled regular expressions.
5. **Defect State Transition:** The database transaction identifies defect #2, verifies it is not already closed, and updates its state to `IN_TESTING`.
6. **Audit Logging:** An immutable audit record is inserted into `audit_logs` documenting the commit hash and state change.
7. **Live SQL Aggregation:** When administrators open the Analytics workspace, FastAPI executes SQL aggregations computing live defect distributions and updated quality metrics.
8. **Plotly Visualization:** The React frontend receives the aggregated JSON coordinates and updates the 14-day trend line and workflow pipeline bar charts in real time.

12 Technical Implementation

Tables 2 and 3 summarize the technical architecture, library versions, and subsystem responsibilities implemented in Milestone 3.

13 Results and Outcomes

The implementation completed during Milestone 3 successfully fulfills all four core mandate requirements stipulated by the Infosys Springboard Virtual Internship guidelines. Table 4 provides a direct mapping between requirements, technical implementation, and verification results.

14 Challenges and Solutions

During the development and integration of Milestone 3, several key technical hurdles were encountered and resolved:

1. **Challenge 1: High Latency in Multi-Dimensional Quality Aggregations**
 - *Problem:* Initial implementations executed multiple independent queries to count open issues, resolved issues, critical defects, and reopen occurrences, introducing high latency ($> 350\text{ms}$) under large ticket volumes.
 - *Solution:* Refactored the calculation into single-pass conditional aggregation queries using SQLAlchemy `case()` statements, executing all counts and sums within one PostgreSQL query round-trip and reducing response latency to under 30ms.

Table 2: BugTracker Core Technology Stack Matrix.

Layer / Domain	Technology	Version / Operational Scope
Frontend Framework	React (TypeScript)	v19.2.8 (Component-driven single page architecture)
Build Tool	Vite	v5.4.19 (High-speed ESM development and bundling)
Frontend Routing	React Router DOM	v7.18.2 (Client-side role-guarded route trees)
Data Visualization	Plotly.js / react-plotly	v4.0.0 / v4.1.0 (Interactive defect trend dashboards)
Charting Library	Recharts	v3.10.1 (Responsive status and priority charts)
PDF Export Engine	jsPDF + AutoTable	v4.2.1 / v5.0.8 (Client-side audit report generation)
HTTP Client	Axios	v1.20.0 (JWT interceptors and API services)
Backend Framework	FastAPI (Python)	v0.141.1 (High-performance asynchronous REST core)
ASGI Web Server	Uvicorn	v0.52.4 (Asynchronous HTTP/WebSocket worker server)
Relational Database	PostgreSQL	v16.x (Relational storage with JSONB audit support)
Database ORM	SQLAlchemy (AsyncIO)	v2.0.52 (Asynchronous ORM sessions and expressions)
Database Drivers	asyncpg / psycopg	v0.31.0 / v3.3.4 (High-speed asynchronous PostgreSQL drivers)
Schema Serialization	Pydantic	v2.13.4 (Strict DTO validation and typing)
Authentication	PyJWT + bcrypt	v2.13.0 / v5.0.0 (HS256 tokens and salted password hashing)
Test Automation	Pytest + pytest-asyncio	v9.1.1 (AsyncIO testing suite, 422 passing tests)
Webhook Relay	Smee Client	v2.0.x (Local development webhook forwarding)

Table 3: Analytics and Integration Subsystem Modules.

Module Path	Primary Purpose	Key Responsibilities and Capabilities
<code>routes/analytics.py</code>	Analytics HTTP End-points	Declares 13 REST routes for system overview, distributions, developer KPIs, and Plotly data bundles.
<code>services/analytics_service.py</code>	Analytical Query Engine	Implements single-pass SQL aggregations for Fix Rate, MTTR, Defect Leakage, and Backlog Health.
<code>routes/webhooks.py</code>	Git Webhook Ingress	Consumes external GitHub push events, extracts commit references, and auto-transitions defects.
<code>routes/export.py</code>	Data Export Gateway	Streams RFC 4180 CSV files and exports consolidated JSON data bundles for client-side PDF rendering.
<code>components/PlotlyDashboard.tsx</code>	Interactive Visualizer	Renders 14-day defect trends, severity donut, and pipeline stage bar charts using React Plotly.
<code>pages/AnalyticsPage.tsx</code>	Central Analytics Console	Tabbed administrative interface providing overview metrics, quality KPIs, and Plotly dashboards.

Table 4: Milestone 3 Requirement Fulfillment Matrix.

Milestone 3 Requirement	Technical Implementation in BugTracker	Verification	Status
1. Deploy REST APIs for issue management and reporting operations	Implemented 24+ asynchronous FastAPI endpoints with Pydantic v2 schemas, JWT authentication, RBAC guards, and self-documenting Swagger/OpenAPI UI	Tested via 422 Pytest tests and Swagger Explorer	Completed
2. Implement project analytics and software quality monitoring modules	Engineered SQL-aggregated quality engine computing Fix Rate, MTTR, Defect Leakage, and Backlog Health Score from live PostgreSQL data	Verified in <code>test_analytics.py</code> and Analytics dashboard	Completed
3. Configure GitHub integration and CI/CD synchronization workflows	Deployed <code>/webhooks/git</code> endpoint with Smee.io payload relay, commit parsing (<code>fixes #2</code> , <code>closes #5</code>), status transitions, and audit logging	Verified with live GitHub pushes and audit traces	Completed
4. Develop administrative dashboards with defect trend visualization	Constructed React Plotly.js dashboard rendering 14-day defect trends, severity donut, and workflow pipeline bar charts	Transpiled via Vite with zero errors; verified in UI	Completed

2. Challenge 2: Ingress Delivery of Webhooks in Local Development

- *Problem:* External GitHub push webhooks cannot directly reach local host listening ports behind private NAT routers without complex network configurations.
- *Solution:* Integrated the open-source Smee.io payload proxy client, establishing a persistent HTTPS tunnel that relays GitHub push events directly to the local FastAPI server with zero firewall modifications.

3. Challenge 3: Dual Reference Syntax in Commit Parsing

- *Problem:* Developers reference tickets using both numeric syntax (`fixes #2`) and project key syntax (`fixes BUG-102`). Naive regex matching frequently resulted in duplicate transitions or false negatives.
- *Solution:* Developed a multi-stage regex tokenizer that isolates numeric IDs and alphanumeric keys into distinct sets, validates existence against database indexes, and deduplicates targets prior to executing the state mutation.

4. Challenge 4: Plotly.js Bundle Size and React 19 Hydration Compatibility

- *Problem:* The full Plotly.js distribution exceeds 3.5MB, causing noticeable bundle bloat and type mismatches with React 19.
- *Solution:* Adopted `plotly.js-dist-min` paired with `react-plotly.js`, applying component-level lazy loading and containerized style overrides to achieve seamless dark-mode rendering with minimal bundle overhead.

15 Conclusion

Milestone 3 successfully transforms BugTracker from a transactional issue tracking system into a mature, API-driven defect intelligence and analytics platform. By combining standardized RESTful APIs, OpenAPI/Swagger documentation, live empirical software quality metrics, bidirectional GitHub push event synchronization, and interactive Plotly defect trend visualizations, the system delivers an enterprise-grade engineering workspace.

The platform provides engineering leaders with complete transparency into defect resolution velocity, guarantees audit compliance through immutable activity logs, and automates developer workflows by linking code commits directly to defect resolution states. The successful verification of 422 automated backend tests and clean frontend build execution demonstrates the technical robustness, architectural integrity, and production readiness of the BugTracker system.

16 References & Technologies Used

1. FastAPI Framework Documentation – Asynchronous Web APIs with Python (<https://fastapi.tiangolo.com/>)
2. PostgreSQL 16 Documentation – Relational Database Management System (<https://www.postgresql.org/docs/16/>)
3. SQLAlchemy 2.0 Unified Documentation – Asynchronous ORM and Database Toolkit (<https://docs.sqlalchemy.org/en/20/>).
4. React 19 Documentation – The Library for Web and Native User Interfaces (<https://react.dev/>).
5. Plotly JavaScript Open Source Graphing Library – Interactive Data Visualization (<https://plotly.com/javascript/>)

6. GitHub REST API and Webhooks Documentation – Event-Driven Integration (<https://docs.github.com>)
7. Smee.io Webhook Payload Delivery Service (<https://smee.io/>).
8. RFC 7519: JSON Web Token (JWT) Standard – Internet Engineering Task Force (<https://datatracker.ietf.org/doc/html/rfc7519>)
9. RFC 4180: Common Format and MIME Type for Comma-Separated Values (CSV) Files (<https://datatracker.ietf.org/doc/html/rfc4180>).
10. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Submitted by:

Ajay Kumar K R – Infosys Springboard Virtual Internship 7.0 (Batch 3)